

What is OOP?

OOP stands for **Object-Oriented Programming**. It's like organizing your code just like you organize real-life things — like cars, students, mobiles — into categories.

Encapsulation

- Wraps **data and methods** into a single unit (class).
 - Uses **access modifiers** (private, public, protected).
 - **Hides internal details**, only exposes what's necessary.
 - Achieved using **getters and setters**.
 - Ensures **data protection and security**.
-

2. Abstraction

- Shows only the **essential features** of an object.
 - **Hides complex internal logic** from the user.
 - Focuses on **what** an object does, not **how**.
 - Achieved using **abstract classes or interfaces**.
 - Increases **code simplicity** and **maintainability**.
-

3. Inheritance

- Allows a class (**child**) to inherit properties/methods of another (**parent**).
 - Promotes **code reuse** and **hierarchical classification**.
 - Supports **single, multiple, multilevel, hierarchical** inheritance.
 - Helps in **extending functionality** without rewriting code.
-

4. Polymorphism

- Means "**many forms**".
- Allows **same method name** to behave differently:

- **Function overloading** (compile-time)
- **Function overriding** (run-time)
- Increases **flexibility** and **extensibility** of code.
- Helps in **dynamic behavior** based on object type.

1. Class – The Blueprint

A **class** is like a design or a plan for something.

➔ Think of a **mobile phone company** that makes many phones. The design (class) is the same, but each phone (object) is different.

Syntax:

```
class ClassName:

    def method_name(self):

        print("Doing something")
```

Example:

```
class Dog:

    def bark(self):

        print(f"{self.name} is barking!")
```

2. Object – Real Thing Created from Class

An **object** is the actual thing made using the class. Like a real dog from the dog design.

Syntax:

```
object_name = ClassName(value)
```

Example:

```
d1 = Dog("Tommy")

d1.bark() # Output: Tommy is barking!
```

What is a Constructor?

A **constructor** is a special function in a class that **automatically runs** when you **create an object**.

Think of it like setting up a new phone — when you buy it, you enter your name, Wi-Fi, etc.

That setup is the constructor.

Why Use a Constructor?

- To **give values** to the object at the time of creation.
- It saves time and ensures the object is **ready to use**.

Python Constructor Syntax

class ClassName:

```
def __init__(self, arg1, arg2):
```

```
    self.arg1 = arg1
```

```
    self.arg2 = arg2
```

- `__init__` is the constructor in Python.
- `self` refers to the current object.
- It runs **automatically** when you create the object

Point	Explanation
Name of constructor	<code>__init__</code>
Auto-run	Runs automatically when object is created
Purpose	To initialize object variables
Use self	To access object's own variables and methods

Self Parameter

Self parameter is a reference to the current instance of the class. It allows us to access the attributes and methods of the object.

class Dog:

```
def __init__(self, name, age):
```

```
    self.name = name
```

```
    self.age = age
```

```
def bark(self):
```

```
    print(f"{self.name} is barking!")
```

3. Methods – Things an Object Can Do

Methods are like actions. If an object is a mobile, the method can be "call" or "click photo".

Syntax:

```
class ClassName:  
    def method_name(self):  
        print("This is a method")
```

Full Example:

```
class Student:  
    def __init__(self, name, grade):  
        self.name = name  
        self.grade = grade  
  
    def show(self):  
        print(f"{self.name} is in grade {self.grade}.")  
  
s1 = Student("Aditya", 12)  
s1.show()
```

Concept	Simple Meaning	Python Keyword	Syntax Example
Class	Design/Plan	class	class Car:
Object	Real thing made from class	(no keyword)	mycar = Car("BMW")
Method	Action it can do	def	def drive(self):

Example:

```
class Student:  
    def __init__(self, name, grade):  
        self.name = name  
        self.grade = grade
```

```
def show_info(self):

    print(f"Name: {self.name}")

    print(f"Grade: {self.grade}")

s1 = Student("Aditya", 12)

s1.show_info()
```

Part	What it does
class Student:	Creates a class called Student
__init__	Constructor – sets name and grade
show_info	Method – prints details of the student
s1 = Student(...)	Creates an object
s1.show_info()	Calls the method on the object

What is Inheritance?

Inheritance lets a **child class** use the **properties and methods** of a **parent class** — like a son inheriting money or habits from a father.

```
class Parent:
```

```
    def speak(self):

        print("Speaking...")
```

```
class Child(Parent): # Inheriting from Parent
```

```
    def walk(self):

        print("Walking...")
```

```
c = Child()
```

```
c.speak() # From Parent
```

```
c.walk() # From Child
```

1. Single Inheritance

One child inherits from one parent.

```
class Parent:
```

```
    def show(self):
```

```
        print("I am the Parent")
```

```
class Child(Parent):
```

```
    def display(self):
```

```
        print("I am the Child")
```

```
c = Child()
```

```
c.show()    # From Parent
```

```
c.display() # From Child
```

Multiple Inheritance

One child inherits from **more than one parent**.

```
class Father:
```

```
    def skill1(self):
```

```
        print("Knows driving")
```

```
class Mother:
```

```
    def skill2(self):
```

```
        print("Knows cooking")
```

```
class Child(Father, Mother):
```

```
    pass
```

```
c = Child()
```

```
c.skill1()
```

```
c.skill2()
```

3. Multilevel Inheritance

Grandchild inherits from child who inherits from parent.

```
class Grandparent:
```

```
    def house(self):
```

```
        print("Has a big house")
```

```
class Parent(Grandparent):
```

```
    def car(self):
```

```
        print("Has a car")
```

```
class Child(Parent):
```

```
    def bike(self):
```

```
        print("Has a bike")
```

```
c = Child()
```

```
c.house()
```

```
c.car()
```

```
c.bike()
```

4. Hierarchical Inheritance

One parent, multiple children.

```
class Parent:
```

```
def speak(self):  
    print("Parent speaks")
```

```
class Child1(Parent):  
    def play(self):  
        print("Child1 plays")
```

```
class Child2(Parent):  
    def dance(self):  
        print("Child2 dances")
```

```
c1 = Child1()
```

```
c2 = Child2()
```

```
c1.speak()
```

```
c2.speak()
```

5. Hybrid Inheritance

Combination of more than one type. Python handles this using **MRO (Method Resolution Order)**.

```
class A:  
    def show(self):  
        print("A class")
```

```
class B(A):  
    pass
```

```

class C(A):

    def show(self):

        print("C class")


class D(B, C): # Inherits from B and C (Hybrid)

    pass


d = D()

d.show()

```

Type	Structure	Example Classes
Single	$A \rightarrow B$	Parent $\rightarrow$ Child
Multiple	$A, B \rightarrow C$	Father, Mother $\rightarrow$ Child
Multilevel	$A \rightarrow B \rightarrow C$	Grandparent $\rightarrow$ Parent $\rightarrow$ Child
Hierarchical	$A \rightarrow B, C$	Teacher $\rightarrow$ Student1, Student2
Hybrid	Combo of above types	$A \rightarrow B, C \rightarrow D$

What is Encapsulation?

Encapsulation is the process of **hiding internal data** of a class and only allowing access through **methods**.

Type	Modifier	Access Level	Example
Public	No underscore	Accessible from anywhere	self.name
Protected	<code>_single</code>	Accessible in class & subclasses	self._name
Private	<code>__double</code>	Accessible only in class	self.__name

1. Public (Default)

```

class Student:

    def __init__(self):

        self.name = "Aditya" # public

```

```
s = Student()
```

```
print(s.name) #  accessible
```

2. Protected (Single underscore _)

```
class Student:
```

```
    def __init__(self):
```

```
        self._grade = "A" # protected
```

```
s = Student()
```

```
print(s._grade) # technically accessible but not recommended
```

3. Private (Double underscore __)

```
class Student:
```

```
    def __init__(self):
```

```
        self.__marks = 95 # private
```

```
    def get_marks(self):
```

```
        return self.__marks
```

```
s = Student()
```

```
print(s.get_marks())# 
```

```
Good print(s.__marks)    #
```

+ Error

Example: Employee Class with All Types

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name        # Public
```

```
        self._department = "IT" # Protected
```

```
        self.__salary = salary  # Private
```

```
    def show_details(self):
```

```
print("Name:", self.name)

print("Department:", self._department)

print("Salary:", self.__salary) # Accessing private within class
```

```
def get_salary(self):

    return self.__salary
```

```
emp = Employee("Aditya", 50000)

# Public - directly accessible

print(emp.name) # ■■■

# Protected - accessible but not recommended

print(emp._department) # 1. Allowed but avoid direct use

# Private - cannot access directly

# print(emp.__salary) + Will give error

# Correct way to access private data

print(emp.get_salary()) # ■■■ Using method

emp.show_details()
```

What is Polymorphism?

Polymorphism comes from the Greek words *poly* (many) and *morph* (forms), meaning **"many forms."**

In programming, polymorphism allows **one interface to be used for different data types or classes.**

In Python, it means:

- A **single function, operator, or method** can behave **differently** depending on the **object** it is acting upon.

Why Use Polymorphism?

- Helps in **code reusability**
 - Allows **flexibility** and **scalability**
 - Enables working with different object types in a **generic way**
 - Central to **Object-Oriented Programming (OOP)** principles
-

Types of Polymorphism in Python:

1. **Duck Typing**
 2. **Operator Overloading**
 3. **Method Overloading (Simulated)**
 4. **Method Overriding**
-

Example 1: Duck Typing

Python is a dynamically typed language – it doesn't care about an object's class, only whether it has the right method or behavior.

```
class Duck:
```

```
    def speak(self):  
        print("Quack!")
```

```
class Person:
```

```
    def speak(self):  
        print("Hello!")
```

```
def call_speak(obj):
```

```
    obj.speak()
```

```
call_speak(Duck())  # Quack!
```

```
call_speak(Person()) # Hello!
```

Explanation:

class Duck:

- Defines a **class named Duck**.

```
def speak(self):
```

```
    print("Quack!")
```

- The Duck class has a method speak that prints "Quack!".
-

class Person:

- Defines another class called Person.

```
def speak(self):
```

```
    print("Hello!")
```

- The Person class also has a speak method, but it prints "Hello!".
-

```
def call_speak(obj):
```

```
    obj.speak()
```

- Defines a function call_speak() that takes **any object** as a parameter.
 - It **calls the .speak() method** on that object — it doesn't care about the class of the object.
 - This is **Duck Typing** — "If it speaks, we assume it's a speaker!"
-

```
call_speak(Duck()) # Quack!
```

```
call_speak(Person()) # Hello!
```

- Creates a Duck object and a Person object.
 - Passes both to the same function call_speak(), which calls .speak() on them.
 - Output depends on the object's class:
 - Duck() → prints "Quack!"
 - Person() → prints "Hello!"
-

Operator Overloading

Python allows built-in operators to behave differently based on data types.

```
class Book:

    def __init__(self, pages):

        self.pages = pages


    def __add__(self, other):

        return Book(self.pages + other.pages)


    def __str__(self):

        return f'Total pages: {self.pages}'


book1 = Book(100)

book2 = Book(150)

book3 = book1 + book2


print(book3) # Total pages: 250
```

Explanation:

class Book:

- Creates a class Book to represent a book object.
-

```
def __init__(self, pages):

    self.pages = pages
```

- The constructor (`__init__`) takes pages and stores it as an attribute.
 - `book1 = Book(100)` → book1 has 100 pages.
-

```
def __add__(self, other):

    return Book(self.pages + other.pages)
```

- This is **operator overloading**.

- The + operator is redefined for Book objects using __add__.
 - It **adds the pages** of two books and returns a new Book.
-

```
def __str__(self):  
    return f"Total pages: {self.pages}"
```

- __str__ is used when you print() the object.
 - It formats the object output as "Total pages: X".
-

```
book1 = Book(100)
```

```
book2 = Book(150)
```

- Creates two Book objects.

```
book3 = book1 + book2
```

- Adds book1 and book2 using overloaded +, returns new Book with 250 pages.

```
print(book3) # Total pages: 250
```

- Calls __str__() → prints the total pages.
-

Method Overriding

When a **child class** defines a method with the **same name** as in its **parent class**, it **overrides** the parent method.

```
class Animal:
```

```
    def speak(self):  
        print("Animal speaks")
```

```
class Dog(Animal):
```

```
    def speak(self):  
        print("Dog barks")
```

```
class Cat(Animal):
```

```
    def speak(self):
```

```
print("Cat meows")
```

```
def make_it_speak(animal):
```

```
    animal.speak()
```

```
make_it_speak(Animal()) # Animal speaks
```

```
make_it_speak(Dog())   # Dog barks
```

```
make_it_speak(Cat())   # Cat meows
```

Explanation:

```
class Animal:
```

```
    def speak(self):
```

```
        print("Animal speaks")
```

- Animal class defines a `speak()` method.
- This is the **base (parent)** method.

```
class Dog(Animal):
```

```
    def speak(self):
```

```
        print("Dog barks")
```

- Dog class inherits from `Animal`.
- **Overrides** the `speak()` method to provide its own behavior.

```
class Cat(Animal):
```

```
    def speak(self):
```

```
        print("Cat meows")
```

- Another subclass that overrides `speak()`.

```
def make_it_speak(animal):
```

animal.speak()

- A function that accepts any object and calls speak() on it.
- Based on the **actual class**, different outputs will be printed.

```
make_it_speak(Animal()) # Animal speaks
```

```
make_it_speak(Dog()) # Dog barks
```

```
make_it_speak(Cat()) # Cat meows
```

- This is **runtime polymorphism**.
- The speak() method call is **resolved at runtime** based on object type.

Recap for Students:

Concept	Python Feature	Key Example
Duck Typing	Dynamic typing	obj.speak() works for Duck/Person
Operator Overloading	<code>add</code> , <code>str</code>	Book example
Method Overloading	Default args, *args	hello(name=None)
Method Overriding	Inheritance & Override	Dog(Animal).speak()

What is Abstraction?

Abstraction means **hiding complex details** and **showing only the necessary features** to the user.

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC): # Abstract Base Class
    @abstractmethod
    def area(self):
        pass
```

```
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
```

```
def area(self): # Overriding the abstract method
    return 3.14 * self.radius * self.radius
```

```
c = Circle(5)
```

```
print(c.area()) # Output: 78.5
```

EXPLANATION:

```
from abc import ABC, abstractmethod
```

- This line imports **ABC** and **abstractmethod** from Python's **abc module**.
 - **ABC = Abstract Base Class**, which allows defining a blueprint for other classes.
 - **abstractmethod** = used to **declare a method that must be implemented** by all subclasses.
-

◆ **class Shape(ABC):**

- Defines a **class named Shape**, which **inherits from ABC**.
 - This makes Shape an **abstract class** — it **cannot be instantiated directly**.
 - Used as a **blueprint** for other shapes like Circle, Rectangle, etc.
-

◆ **@abstractmethod**

- This **decorator** is used to mark the method below as **abstract**.
 - It **forces subclasses** to implement this method.
 - If a subclass does **not override** it, Python will raise an **error**.
-

◆ **def area(self):**

- This is an **abstract method** of the Shape class.
 - It does **not contain any logic** here, just a placeholder.
 - The method must be **implemented by all subclasses** (like Circle).
-

◆ **pass**

- Placeholder statement — does **nothing**.
 - Used because Python requires some code inside the function body.
 - This function is just a **declaration**, not implementation.
-

◆ **class Circle(Shape):**

- Defines a **subclass Circle** that inherits from the abstract class Shape.
 - It is **required to implement the area() method**, or else Python will raise a TypeError.
-

◆ **def __init__(self, radius):**

- The **constructor method** of the Circle class.
 - Takes one argument radius and stores it as an **instance variable** self.radius.
-

◆ **self.radius = radius**

- Assigns the input radius to the object's property self.radius for later use.
-

◆ **def area(self):**

- This is the **implementation** of the abstract area() method from Shape.
 - Because Circle has a specific way to calculate area, it overrides the base class method.
-

◆ **return 3.14 * self.radius * self.radius**

- Returns the **area of the circle** using the formula πr^2 ($\pi \approx 3.14$).
- Uses the value stored in self.radius.